RETO PRÁCTICO

Módulo 5 — QA Cloud-Native: Automatización Inteligente con IA Generativa

Fase 1 — Datos de prueba realistas (10 PTS)

Genera y controla los datos del catálogo de la tienda:

- Genera el catálogo con Faker y semilla fija (datos reproducibles en toda máquina).
- Define al menos 4 categorías y 20+ productos con campos coherentes (nombre, precio, stock, categoría).
- Reto extra: parte de un 'dump real' ficticio y aplica anonimización (correos, nombres) antes de usarlo.

Entregable de la fase: Script generar-datos.js con semilla fija + db.json resultante. Explica por qué NO versionas datos generados (los regenera el pipeline).

Fase 2 — Pipeline CI/CD en Azure con quality gates (15 PTS)

Automatiza build, smoke y carga en Azure Pipelines; detén el deploy si algo falla:

- Crea un azure-pipelines.yml con al menos 3 stages: Build -> SmokeTests -> LoadTest.
- El smoke test valida que la app 'encendió' (health, catálogo, crear pedido).
- La prueba de carga evalúa un SLA (p95 < 800 ms y error < 1%) y actúa como quality gate (exit 1 detiene el deploy).
- Demuestra el gate en ROJO forzando el fallo (baja el SLA) y luego en VERDE.
- Conecta el repo de GitHub a Azure Pipelines como origen del código.

Entregable de la fase: El archivo azure-pipelines.yml + capturas de una corrida en verde y una en rojo (gate de carga). Indica dónde está cada gate.

Fase 3 — Del caso manual al automatizado con IA (20 PTS)

Gestiona los casos en Azure Test Plans, tradúcelos a Gherkin y automatízalos con Screenplay + MCP (Claude):

- Define los 3 casos (Catálogo, Carrito, Pedidos) como casos manuales en Azure Test Plans.
- Tradúcelos a Gherkin (keywords en inglés, texto en español) y automatízalos con Playwright + patrón Screenplay + pytest-bdd.
- Usa Claude + servidor MCP de Playwright para generar/mantener al menos 1 escenario nuevo (ej. un caso de borde) y documenta el prompt usado.
- Vincula los casos automatizados de vuelta a Azure Test Plans (descripción/Steps + publicar resultados JUnit en el pipeline).
- Mínimo: 4 escenarios en verde. Reto extra: agrega 2 casos de borde propuestos con IA.

Entregable de la fase: Carpeta e2e/ con features y steps + salida de pytest en verde + captura del escenario que propuso la IA (con el prompt).

Fase 4 — Ejecución en la nube (10 PTS)

Lleva las mismas pruebas a navegadores/dispositivos reales e intégralo al pipeline de Azure:

- Prepara el framework para correr contra Sauce Labs (o equivalente) cambiando solo configuración (BASE_URL / capabilities).
- Integra la ejecución en el azure-pipelines.yml (un stage que dispare las pruebas en la nube).
- Demo guiada: si no hay cuenta, evidencia con capturas la configuración y el código listo para conectarse.

Entregable de la fase: Configuración de ejecución remota + evidencia (captura del dashboard o del código/capabilities preparados). Explica qué cambia frente a local.

Fase 5 — Contenedores y contract testing (15 PTS)
Conteneriza la tienda y valida el contrato de la API (publica la imagen en Azure):

- Escribe un Dockerfile que empaquete la app (incluida la generación de datos) y exponga /api/health como healthcheck.
- Construye y corre la imagen; demuestra que la app responde igual contenerizada.
- Publica la imagen en Azure Container Registry (ACR) y, opcional, despliégala en Azure App Service / Container Apps.
- Implementa un contract test con Pact entre el front (consumer) y la API (provider) para al menos GET /api/productos.
- Reto extra: argumenta si conviene partir el monolito en microservicios y qué contratos aparecerán.

Entregable de la fase: Dockerfile + evidencia de la imagen corriendo (y en ACR) + el pact generado y la verificación provider/consumer en verde.

Fase 6 — Métricas y estrategia (20 PTS)
Mide lo que construiste y argumenta tu estrategia:

- Reporta las métricas DORA aplicadas a tu flujo: frecuencia de despliegue, lead time, tasa de fallo de cambios, MTTR (estimados razonados valen).
- Dibuja dónde cae cada herramienta en la pirámide de pruebas (unit/integration/E2E) y justifica.
- Lista 3 riesgos de calidad de tu solución y cómo los mitigarías.

Entregable de la fase: Cuadro de métricas DORA + diagrama de la pirámide + lista de riesgos/mitigaciones en el documento de soporte.

Entrega final
No hay sustentación en vivo. La entrega consiste en:

- Un **DOCUMENTO DE SOPORTE** (Word o PDF) que explique las decisiones y muestre las evidencias (capturas) de cada fase.
- El **CÓDIGO** publicado en un repositorio de GitHub, con un README que explique cómo correr todo.

Qué debe contener el documento de soporte:

- Una sección por fase (1 a 6) con su entregable y las capturas que lo evidencian.
- El enlace al repositorio de GitHub.
- El cuadro de métricas DORA y la pirámide de pruebas (Fase 6).
- Declaración honesta de qué corre de verdad y qué es demo guiada (Sauce, portales de Azure).

Estructura mínima del repositorio de GitHub:

- app-tienda/ — la app bajo prueba.
- datos/ — script de generación de datos (Faker) y, si aplica, anonimización.
- e2e/ — framework Playwright + Screenplay + BDD (features, steps, screenplay).
- pipeline/ — azure-pipelines.yml con los stages y gates.
- Dockerfile y, si llegas, pact/ (contract testing).
- README.md — cómo correr todo, decisiones y el cuadro de métricas.

Resumen de entregables (10 PTS)

Checklist de lo que debe estar en el documento de soporte y en el repositorio:

- **Fase 1:** generar-datos.js (semilla fija) + db.json.
- **Fase 2:** azure-pipelines.yml + capturas del gate de carga en rojo y en verde.
- **Fase 3:** features/ y steps/ + salida de pytest en verde + escenario generado con IA (con el prompt).
- **Fase 4:** configuración de ejecución en la nube + evidencia.
- **Fase 5:** Dockerfile + imagen corriendo (y en ACR) + contract test (Pact) verificado.
- **Fase 6:** cuadro de métricas DORA + pirámide + riesgos y mitigaciones.
- **General:** enlace al repositorio de GitHub y el documento de soporte con todas las capturas.